DBMS MINI PROJECT

SecureBank

Banking Management System

Submitted By

[Your Full Name]

[Partner Name, if any]

B.Tech – Information Technology

[Your College Name, City]

Academic Year: 2026–27

Project Type: DBMS Mini Project

Table of Contents

1. Introduction

2. Problem Statement

3. Objectives

4. ER Diagram

5. Database Schema

6. Technologies Used

7. GitHub Repository and Project Documentation

8. SQL Queries and Features Implemented

9. Screenshots

10. Testing

11. Conclusion

12. Future Scope

Appendix A: Current Database Record Summary

Appendix B: Complete Database Schema (SQL)

1. Introduction

SecureBank is a database-driven web application developed as a DBMS mini project for managing the core operations of a bank. The system provides a centralized relational database for customers, branches, account types, accounts, transactions, loans, EMI payments and an audit log of account status changes.

The application uses PostgreSQL for database management and Flask with Python for the web interface. HTML and CSS provide a simple user interface. An important design decision is that the banking rules are enforced inside the database itself: a stored function validates and applies every transaction, constraints protect data integrity, a trigger maintains an audit trail, and multi-step operations such as transfers run as atomic transactions.

The project applies important DBMS concepts such as relational design, primary and foreign keys, constraints, normalization up to 3NF, sequences, joins, aggregate queries, subqueries, indexing, views, stored functions, triggers and transaction handling with row locking.

2. Problem Statement

Banking involves money, so the data must always be correct. Managing customer, account and transaction records without a structured database causes duplication, inconsistent balances, lost updates when two operations happen together, and no reliable history of what changed. A transfer that debits one account but fails to credit another would be a serious error.

The objective of SecureBank is to provide a structured relational database and a simple web interface where customers and accounts can be managed, money can be deposited, withdrawn and transferred safely, loans can be approved and repaid through EMIs, and every change can be traced.

3. Objectives

- To design a structured relational database for banking operations.
- To store customers, branches, account types, accounts, transactions, loans, EMI payments and audit records.
- To establish relationships between entities using primary and foreign keys.
- To maintain data integrity using NOT NULL, UNIQUE, CHECK, DEFAULT and foreign key constraints (for example: no negative balance, adult customers only, valid phone and IFSC format).
- To reduce redundancy through normalization up to Third Normal Form (3NF).
- To implement banking rules as stored functions (apply_transaction, calc_emi, post_monthly_interest).
- To create a trigger that records account status changes automatically.
- To demonstrate SQL JOINs, GROUP BY with HAVING, subqueries, NOT EXISTS and views.
- To demonstrate atomic transactions using COMMIT, ROLLBACK and row locking.
- To provide a Flask interface for customers, accounts, transactions, loans and analytics.

4. ER Diagram

The ER model of SecureBank consists of eight entities. The relationships are listed below and shown in the diagram.

- Customers (1) → (Many) Accounts
- Branches (1) → (Many) Accounts
- Account_Types (1) → (Many) Accounts
- Accounts (1) → (Many) Transactions
- Accounts (1) → (Many) Audit_Log
- Customers (1) → (Many) Loans
- Accounts (1) → (Many) Loans (account used for disbursal and EMI)
- Loans (1) → (Many) Loan_Payments

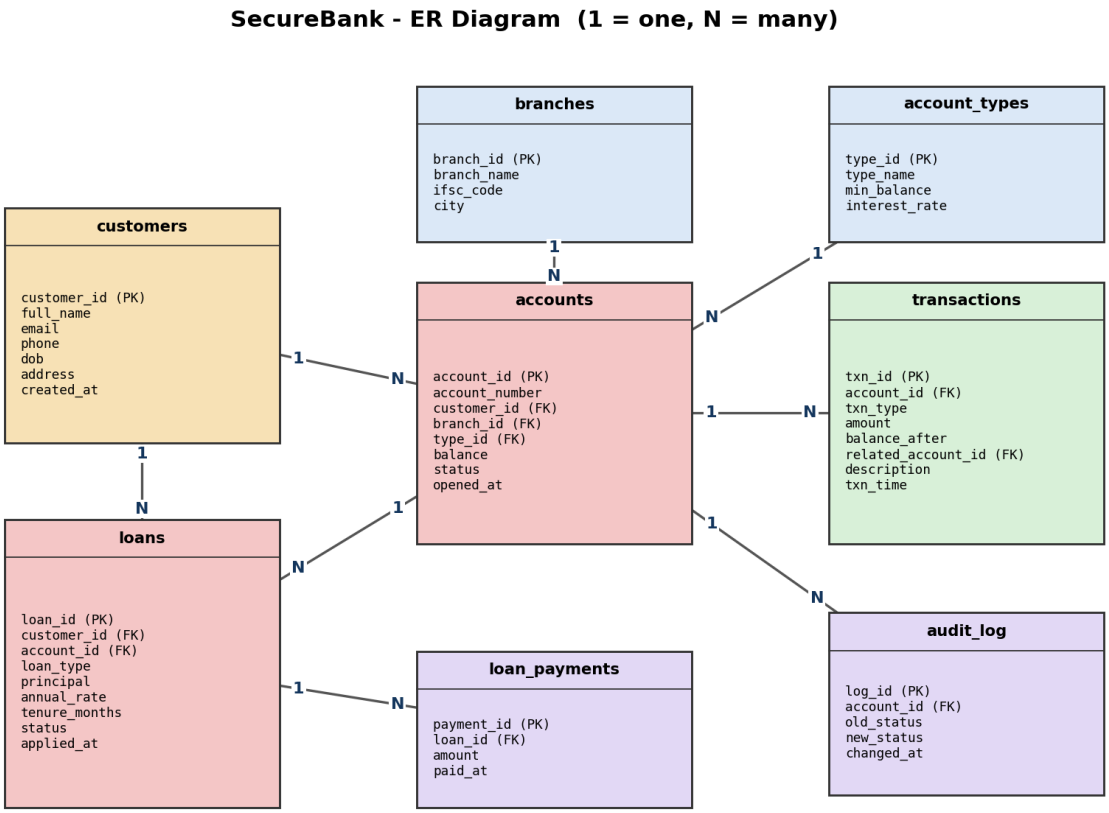


Figure 1: SecureBank ER Diagram

5. Database Schema

The SecureBank database is implemented in PostgreSQL and contains eight relational tables.

Table	Purpose	Key Fields
branches	Stores bank branches and IFSC codes.	branch_id (PK)
customers	Stores customer personal and contact details.	customer_id (PK)
account_types	Stores account categories with minimum balance and interest rate.	type_id (PK)
accounts	Stores bank accounts, balances and status.	account_id (PK), customer_id (FK), branch_id (FK), type_id (FK)
transactions	Stores every deposit, withdrawal, transfer, interest, loan and EMI entry (account statement).	txn_id (PK), account_id (FK), related_account_id (FK)
loans	Stores loan applications and their status.	loan_id (PK), customer_id (FK), account_id (FK)
loan_payments	Stores EMI payments made for loans.	payment_id (PK), loan_id (FK)
audit_log	Stores account status changes (filled by a trigger).	log_id (PK), account_id (FK)

5.1 Relationships

- A customer can hold many accounts (one-to-many); an account belongs to exactly one customer, one branch and one account type.
- An account has many transactions. A transfer creates two transaction rows, linked through related_account_id.
- A customer can have many loans; each loan is tied to one account for disbursal and EMI debits.
- A loan has many EMI payment records.
- An account has many audit log entries recording status changes.

5.2 Data Dictionary

The following tables were generated directly from the PostgreSQL system catalogs of the implemented database.

Table: branches

Column	Data Type	Null?	Default
branch_id	INTEGER	NO	SERIAL (auto)
branch_name	VARCHAR(60)	NO	
ifsc_code	VARCHAR(11)	NO	
city	VARCHAR(40)	NO	

Constraint Type	Definition
CHECK	CHECK (((ifsc_code)::text ~ '^[A-Z]{4}0[A-Z0-9]{6}$'::text))
PRIMARY KEY	PRIMARY KEY (branch_id)
UNIQUE	UNIQUE (branch_name)
UNIQUE	UNIQUE (ifsc_code)

Table: customers

Column	Data Type	Null?	Default
customer_id	INTEGER	NO	SERIAL (auto)
full_name	VARCHAR(80)	NO	
email	VARCHAR(80)	NO	
phone	VARCHAR(10)	NO	
dob	DATE	NO	
address	TEXT	YES	
created_at	TIMESTAMP	NO	now()

Constraint Type	Definition
CHECK	CHECK ((dob <= '2100-01-01'::date))
CHECK	CHECK ((dob <= (CURRENT_DATE - '18 years'::interval)))
CHECK	CHECK (((email)::text ~~ '%_@_%.__%'::text))
CHECK	CHECK (((phone)::text ~ '^[6-9][0-9]{9}$'::text))
PRIMARY KEY	PRIMARY KEY (customer_id)
UNIQUE	UNIQUE (email)
UNIQUE	UNIQUE (phone)

Table: account_types

Column	Data Type	Null?	Default
type_id	INTEGER	NO	SERIAL (auto)
type_name	VARCHAR(20)	NO	
min_balance	NUMERIC	NO	0
interest_rate	NUMERIC	NO	0

Constraint Type	Definition
CHECK	CHECK (((interest_rate >= (0)::numeric) AND (interest_rate <= (15)::numeric)))
CHECK	CHECK ((min_balance >= (0)::numeric))
PRIMARY KEY	PRIMARY KEY (type_id)
UNIQUE	UNIQUE (type_name)

Table: accounts

Column	Data Type	Null?	Default
account_id	INTEGER	NO	SERIAL (auto)
account_number	VARCHAR(12)	NO	sequence account_no_seq
customer_id	INTEGER	NO	
branch_id	INTEGER	NO	
type_id	INTEGER	NO	
balance	NUMERIC	NO	0
status	VARCHAR(10)	NO	'Active'::character varying
opened_at	TIMESTAMP	NO	now()

Constraint Type	Definition
CHECK	CHECK ((balance >= (0)::numeric))
CHECK	CHECK (((status)::text = ANY ((ARRAY['Active'::character varying, 'Frozen'::character varying, 'Closed'::character varying])::text[])))
FOREIGN KEY	FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
FOREIGN KEY	FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE RESTRICT
FOREIGN KEY	FOREIGN KEY (type_id) REFERENCES account_types(type_id)
PRIMARY KEY	PRIMARY KEY (account_id)
UNIQUE	UNIQUE (account_number)

Table: transactions

Column	Data Type	Null?	Default
txn_id	INTEGER	NO	SERIAL (auto)
account_id	INTEGER	NO	
txn_type	VARCHAR(15)	NO	
amount	NUMERIC	NO	
balance_after	NUMERIC	NO	
related_account_id	INTEGER	YES	
description	TEXT	YES	
txn_time	TIMESTAMP	NO	now()

Constraint Type	Definition
CHECK	CHECK ((amount > (0)::numeric))
CHECK	CHECK ((balance_after >= (0)::numeric))
CHECK	CHECK (((txn_type)::text = ANY ((ARRAY['Deposit'::character varying, 'Withdrawal'::character varying, 'Transfer-In'::character varying, 'Transfer-Out'::character varying, 'Interest'::character varying, 'Loan Disbursal'::character varying, 'EMI'::character varying])::text[])))
FOREIGN KEY	FOREIGN KEY (account_id) REFERENCES accounts(account_id) ON DELETE CASCADE
FOREIGN KEY	FOREIGN KEY (related_account_id) REFERENCES accounts(account_id)
PRIMARY KEY	PRIMARY KEY (txn_id)

Table: loans

Column	Data Type	Null?	Default
loan_id	INTEGER	NO	SERIAL (auto)
customer_id	INTEGER	NO	
account_id	INTEGER	NO	
loan_type	VARCHAR(15)	NO	
principal	NUMERIC	NO	
annual_rate	NUMERIC	NO	
tenure_months	INTEGER	NO	
status	VARCHAR(10)	NO	'Pending'::character varying
applied_at	TIMESTAMP	NO	now()

Constraint Type	Definition
CHECK	CHECK ((annual_rate >= (0)::numeric))
CHECK	CHECK (((loan_type)::text = ANY ((ARRAY['Home'::character varying, 'Education'::character varying, 'Personal'::character varying, 'Vehicle'::character varying])::text[])))
CHECK	CHECK ((principal > (0)::numeric))
CHECK	CHECK (((status)::text = ANY ((ARRAY['Pending'::character varying, 'Approved'::character varying, 'Rejected'::character varying, 'Closed'::character varying])::text[])))
CHECK	CHECK (((tenure_months >= 1) AND (tenure_months <= 360)))
FOREIGN KEY	FOREIGN KEY (account_id) REFERENCES accounts(account_id)
FOREIGN KEY	FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
PRIMARY KEY	PRIMARY KEY (loan_id)

Table: loan_payments

Column	Data Type	Null?	Default
payment_id	INTEGER	NO	SERIAL (auto)
loan_id	INTEGER	NO	
amount	NUMERIC	NO	
paid_at	TIMESTAMP	NO	now()

Constraint Type	Definition
CHECK	CHECK ((amount > (0)::numeric))
FOREIGN KEY	FOREIGN KEY (loan_id) REFERENCES loans(loan_id) ON DELETE CASCADE
PRIMARY KEY	PRIMARY KEY (payment_id)

Table: audit_log

Column	Data Type	Null?	Default
log_id	INTEGER	NO	SERIAL (auto)
account_id	INTEGER	NO	
old_status	VARCHAR(10)	YES	
new_status	VARCHAR(10)	NO	
changed_at	TIMESTAMP	NO	now()

Constraint Type	Definition
FOREIGN KEY	FOREIGN KEY (account_id) REFERENCES accounts(account_id) ON DELETE CASCADE
PRIMARY KEY	PRIMARY KEY (log_id)

5.3 Constraints

- Primary keys uniquely identify records in each table (SERIAL auto-increment).
- Foreign keys maintain referential integrity; customers with accounts cannot be deleted (ON DELETE RESTRICT).
- NOT NULL constraints ensure required values are provided.
- UNIQUE constraints prevent duplicate emails, phone numbers, branch names, IFSC codes and account numbers.
- CHECK constraints enforce: balance >= 0, amount > 0, valid status and transaction type values, valid phone (10 digits starting 6-9), email format, IFSC format, customer age of at least 18, loan tenure between 1 and 360 months.
- DEFAULT values provide the account status 'Active', balance 0, timestamps and a generated account number.
- A SEQUENCE (account_no_seq) is used as the DEFAULT of account_number to generate numbers such as AC10000001.
- ON DELETE CASCADE is used for dependent records (transactions, loan payments, audit log).

5.4 Normalization

First Normal Form (1NF): all columns hold atomic values and every table has a primary key. For example, each transaction is its own row rather than a list inside the account record.

Second Normal Form (2NF): every table has a single-column primary key, so no attribute depends on part of a key.

Third Normal Form (3NF): there are no transitive dependencies. Branch details, account type rules (minimum balance, interest rate) and customer details are stored once and referenced by ID, not repeated in every account or transaction.

Note on stored values: accounts.balance and transactions.balance_after could be derived by summing transactions, but they are stored deliberately: the current balance must be read instantly, can be protected by a CHECK constraint and row lock, and balance_after is a historical snapshot printed on the account statement. The stored function apply_transaction() is the only code that changes a balance, which keeps these values consistent with the transaction rows.

6. Technologies Used

Technology	Purpose
Python	Programming language used for the web application.
Flask	Python web framework used to build routes and pages.
PostgreSQL	Relational database management system.
PL/pgSQL	Procedural language used for stored functions and the trigger.
HTML	Used to structure web pages (Jinja2 templates).
CSS	Used to style the web interface.
psycopg2	Python PostgreSQL database adapter.
pgAdmin / psql	Used for PostgreSQL database management and testing.
Git & GitHub	Used for version control and project hosting.

7. GitHub Repository and Project Documentation

The complete SecureBank project is uploaded to GitHub for source-code management, version control and project submission.

GitHub Repository:

https://github.com/[your-username]/SecureBank

The repository contains the complete project files, including:

- Flask application source code (app.py) and automated tests
- HTML templates and CSS stylesheet
- PostgreSQL schema (securebank.sql) and sample data (sample_data.sql)
- setup_db.py script that creates the database and loads everything with one command
- ER diagram image, requirements.txt, .gitignore and README.md
- Database setup and execution instructions

Repository Structure

```
SecureBank/
|-- static/
|   `-- style.css
|-- templates/
|   |-- base.html
|   |-- index.html
|   |-- page.html
|   `-- form.html
|-- tests/
|   `-- test_app.py
|-- docs/
|-- app.py
|-- setup_db.py
|-- securebank.sql
|-- sample_data.sql
|-- er_diagram.png
|-- requirements.txt
|-- .gitignore
`-- README.md
```

Version Control: Git was used for version control during development and GitHub hosts the repository, which serves as the central location for source code and documentation.

8. SQL Queries and Features Implemented

The application uses SQL to retrieve, insert and update data in PostgreSQL. The most important database features are described below.

8.1 Table Creation with Constraints (example: accounts)

```
CREATE TABLE accounts (
    account_id      SERIAL PRIMARY KEY,
    account_number  VARCHAR(12) NOT NULL UNIQUE DEFAULT ('AC' || LPAD(nextval('account_no_seq')::TEXT, 8,
        '0')),
    customer_id     INT NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT,
    branch_id       INT NOT NULL REFERENCES branches(branch_id),
    type_id         INT NOT NULL REFERENCES account_types(type_id),
    balance         NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (balance >= 0),
    status          VARCHAR(10) NOT NULL DEFAULT 'Active' CHECK (status IN ('Active','Frozen','Closed')),
    opened_at       TIMESTAMP NOT NULL DEFAULT NOW()
);
```

8.2 Indexing

Indexes are created on foreign key columns and on the transaction time to speed up joins, statements and date-based queries.

Index Name	Table(column)
idx_accounts_customer	accounts(customer_id)
idx_accounts_branch	accounts(branch_id)
idx_accounts_type	accounts(type_id)
idx_txn_account	transactions(account_id)
idx_txn_time	transactions(txn_time)
idx_loans_customer	loans(customer_id)
idx_loans_account	loans(account_id)
idx_payments_loan	loan_payments(loan_id)
idx_audit_account	audit_log(account_id)

8.3 Stored Function: apply_transaction()

Every change of a balance goes through this PL/pgSQL function. It checks the amount, locks the account row with FOR UPDATE, rejects non-active accounts, enforces the account type's minimum balance for debits, updates the balance and writes the statement row. If a rule is violated it raises an exception, which rolls back the whole transaction.

```
CREATE FUNCTION apply_transaction(p_account INT, p_type VARCHAR, p_amount NUMERIC,
                                  p_desc TEXT DEFAULT NULL, p_related INT DEFAULT NULL)
RETURNS NUMERIC AS $$
DECLARE v_bal NUMERIC; v_status VARCHAR; v_min NUMERIC; v_new NUMERIC;
BEGIN
    IF p_amount IS NULL OR p_amount <= 0 THEN
        RAISE EXCEPTION 'Amount must be greater than zero.';
    END IF;
    SELECT a.balance, a.status, t.min_balance INTO v_bal, v_status, v_min
    FROM accounts a JOIN account_types t ON t.type_id = a.type_id
    WHERE a.account_id = p_account FOR UPDATE OF a;                -- row lock
    IF NOT FOUND THEN RAISE EXCEPTION 'Account not found.'; END IF;
    IF v_status <> 'Active' THEN
        RAISE EXCEPTION 'Account is % and cannot be used for transactions.', v_status;
    END IF;
    IF p_type IN ('Withdrawal','Transfer-Out','EMI') THEN
        v_new := v_bal - p_amount;
        IF v_new < v_min THEN
            RAISE EXCEPTION 'Insufficient funds: minimum balance of Rs. % must be maintained.', v_min;
        END IF;
    ELSE
        v_new := v_bal + p_amount;
    END IF;
    UPDATE accounts SET balance = v_new WHERE account_id = p_account;
    INSERT INTO transactions (account_id, txn_type, amount, balance_after, related_account_id,
        description)
```

```
    VALUES (p_account, p_type, p_amount, v_new, p_related, p_desc);
    RETURN v_new;
END;
$$ LANGUAGE plpgsql;
```

8.4 Stored Functions: calc_emi() and post_monthly_interest()

calc_emi() computes the EMI with the standard formula EMI = P x r x (1+r)^n / ((1+r)^n - 1), where r is the monthly rate. post_monthly_interest() loops over active interest-bearing accounts and credits interest through apply_transaction().

```
-- EMI = P * r * (1+r)^n / ((1+r)^n - 1), r = monthly rate
CREATE FUNCTION calc_emi(p NUMERIC, annual_rate NUMERIC, months INT) RETURNS NUMERIC AS $$
    SELECT CASE WHEN annual_rate = 0 THEN ROUND(p / months, 2)
           ELSE ROUND(p * (annual_rate/1200) * POWER(1 + annual_rate/1200, months)
                      / (POWER(1 + annual_rate/1200, months) - 1), 2) END;
$$ LANGUAGE sql IMMUTABLE;

-- Credits monthly interest to every active account whose type earns interest
CREATE FUNCTION post_monthly_interest() RETURNS INT AS $$
DECLARE r RECORD; n INT := 0; v_int NUMERIC;
BEGIN
    FOR r IN SELECT a.account_id, a.balance, t.interest_rate
             FROM accounts a JOIN account_types t ON t.type_id = a.type_id
             WHERE a.status = 'Active' AND t.interest_rate > 0 LOOP
        v_int := ROUND(r.balance * r.interest_rate / 1200, 2);
        IF v_int > 0 THEN
            PERFORM apply_transaction(r.account_id, 'Interest', v_int, 'Monthly interest');
            n := n + 1;
        END IF;
    END LOOP;
    RETURN n;
END;
$$ LANGUAGE plpgsql;
```

8.5 Trigger: Account Status Audit

A trigger records the opening of every account and every later change of its status (Active, Frozen, Closed). The application never writes to audit_log directly.

```
-- Trigger: audit trail for account status (opened / frozen / unfrozen / closed)
CREATE FUNCTION log_account_status() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log (account_id, old_status, new_status) VALUES (NEW.account_id, NULL,
            NEW.status);
    ELSIF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO audit_log (account_id, old_status, new_status) VALUES (NEW.account_id, OLD.status,
            NEW.status);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_account_status AFTER INSERT OR UPDATE OF status ON accounts
FOR EACH ROW EXECUTE FUNCTION log_account_status();
```

8.6 Views

```
CREATE VIEW v_account_summary AS
SELECT a.account_id, a.account_number, c.full_name, t.type_name, b.branch_name, a.balance, a.status,
    a.opened_at
FROM accounts a
JOIN customers c ON c.customer_id = a.customer_id
JOIN account_types t ON t.type_id = a.type_id
JOIN branches b ON b.branch_id = a.branch_id;

CREATE VIEW v_branch_summary AS
SELECT b.branch_name, b.city, COUNT(a.account_id) AS accounts, COALESCE(SUM(a.balance), 0) AS
    total_balance
FROM branches b LEFT JOIN accounts a ON a.branch_id = b.branch_id
GROUP BY b.branch_id;

CREATE VIEW v_loan_summary AS
SELECT l.loan_id, c.full_name, a.account_number, l.loan_type, l.principal, l.annual_rate, l.tenure_months,
       calc_emi(l.principal, l.annual_rate, l.tenure_months) AS emi, l.status,
       COUNT(p.payment_id) AS emis_paid, l.applied_at
FROM loans l
```

```
JOIN customers c ON c.customer_id = l.customer_id
JOIN accounts a ON a.account_id = l.account_id
LEFT JOIN loan_payments p ON p.loan_id = l.loan_id
GROUP BY l.loan_id, c.full_name, a.account_number;
```

8.7 JOIN, Aggregate, Subquery and NOT EXISTS Queries

```
-- Customers with their account count and total balance (LEFT JOIN + GROUP BY)
SELECT c.customer_id, c.full_name, c.email, c.phone, COUNT(a.account_id), COALESCE(SUM(a.balance),0)
FROM customers c LEFT JOIN accounts a ON a.customer_id = c.customer_id GROUP BY c.customer_id ORDER BY 1;

-- Recent transactions (3-table JOIN)
SELECT t.txn_id, a.account_number, c.full_name, t.txn_type, t.amount, t.balance_after, t.txn_time
FROM transactions t JOIN accounts a ON a.account_id = t.account_id
JOIN customers c ON c.customer_id = a.customer_id ORDER BY t.txn_id DESC LIMIT 8;

-- Top customers by total balance (SUM + GROUP BY + LIMIT)
SELECT c.full_name, COUNT(*), SUM(a.balance) FROM customers c JOIN accounts a ON a.customer_id =
    c.customer_id
GROUP BY c.customer_id ORDER BY 3 DESC LIMIT 5;

-- Customers without any account (NOT EXISTS)
SELECT customer_id, full_name, phone FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM accounts a WHERE a.customer_id = c.customer_id);

-- Transactions above the average amount (subquery)
SELECT t.txn_id, a.account_number, t.txn_type, t.amount FROM transactions t
JOIN accounts a ON a.account_id = t.account_id
WHERE t.amount > (SELECT AVG(amount) FROM transactions) ORDER BY t.amount DESC LIMIT 6;

-- Monthly summary by type (GROUP BY + HAVING)
SELECT to_char(txn_time, 'YYYY-MM'), txn_type, COUNT(*), SUM(amount) FROM transactions
GROUP BY 1, 2 HAVING COUNT(*) >= 1 ORDER BY 1 DESC, 4 DESC;
```

8.8 Atomic Transfer (Transaction Handling)

A transfer consists of a debit and a credit. The code locks both account rows in a fixed order (lowest ID first) to avoid deadlocks, then calls apply_transaction() twice. The helper tx() wraps the work in one database transaction: COMMIT if everything succeeds, ROLLBACK if any step raises an error, so money is never debited without being credited.

```
@app.route("/transfer", methods=["GET", "POST"])
def transfer():
    if request.method == "POST":
        def work(cur):
            a, b, amt = int(request.form["from_id"]), int(request.form["to_id"]), amount()
            if a == b:
                raise ValueError("Source and destination accounts must be different.")
            cur.execute("SELECT account_id, account_number FROM accounts WHERE account_id IN (%s, %s)
                ORDER BY account_id FOR UPDATE", (a, b))
            no = dict(cur.fetchall())          # locks both rows in a fixed order (prevents deadlock)
            note = request.form["description"].strip()
            cur.execute("SELECT apply_transaction(%s, 'Transfer-Out', %s, %s, %s)", (a, amt, f"Transfer
                to {no[b]}. {note}".strip(), b))
            cur.execute("SELECT apply_transaction(%s, 'Transfer-In', %s, %s, %s)", (b, amt, f"Transfer
                from {no[a]}. {note}".strip(), a))
            return f"{money(amt)} transferred from {no[a]} to {no[b]}."
        msg = tx(work)
        if msg:
            flash(msg, "ok")
            return redirect(url_for("transfer"))
```

8.9 Other Multi-Table Transactions

Opening an account inserts the account and the opening deposit together. Approving a loan updates the loan status and credits the principal to the customer's account. Paying an EMI debits the account, inserts a loan_payments row and closes the loan after the last EMI. Each of these runs in a single transaction.

8.10 Implemented Application Operations

The application implements Create (customers, accounts, transactions, loans), Read (all list, statement and analytics pages) and Update (account freeze/unfreeze, loan approval, balances

through transactions) operations. Delete operations are not provided in the web interface because banking records should be preserved; ON DELETE rules protect the data.

8.11 Features Not Implemented

User authentication and CSRF protection are not implemented because this is an academic demonstration. Scheduled (automatic) interest posting is not implemented; the interest function is run manually from the home page.

9. Screenshots

The following figures show the application and database output while running the project with the sample data. The scenario shows a deposit, an invalid withdrawal, the minimum balance rule, a successful transfer, a transfer to a frozen account that is rolled back, an account statement, loan approval with EMI payment, monthly interest posting and the analytics page.

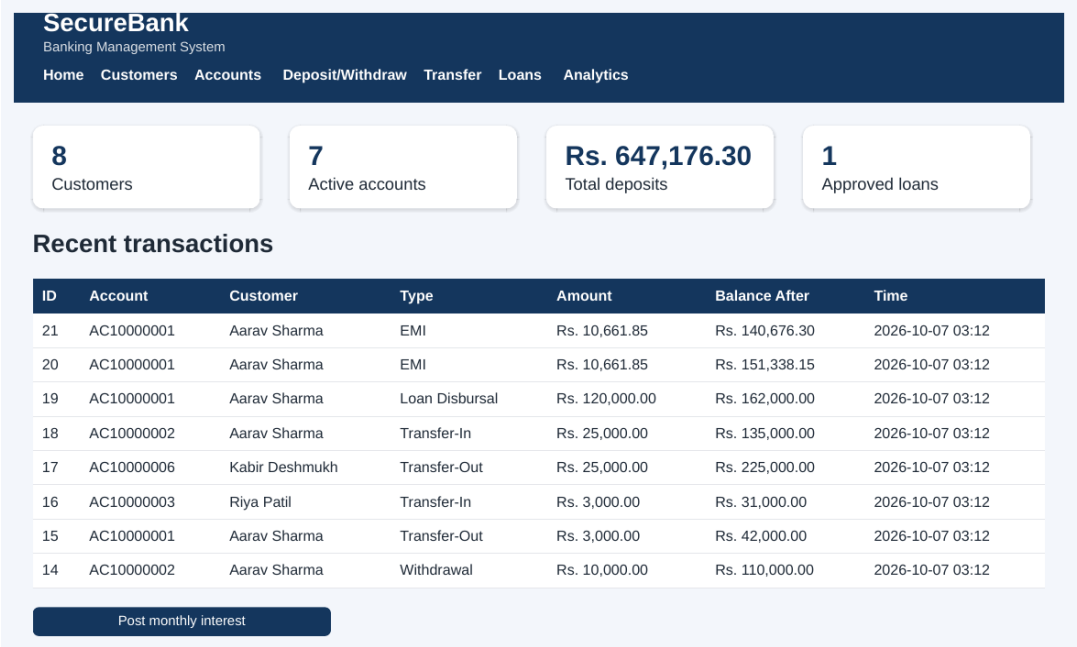


Figure 2: SecureBank Home Page (dashboard and recent transactions)

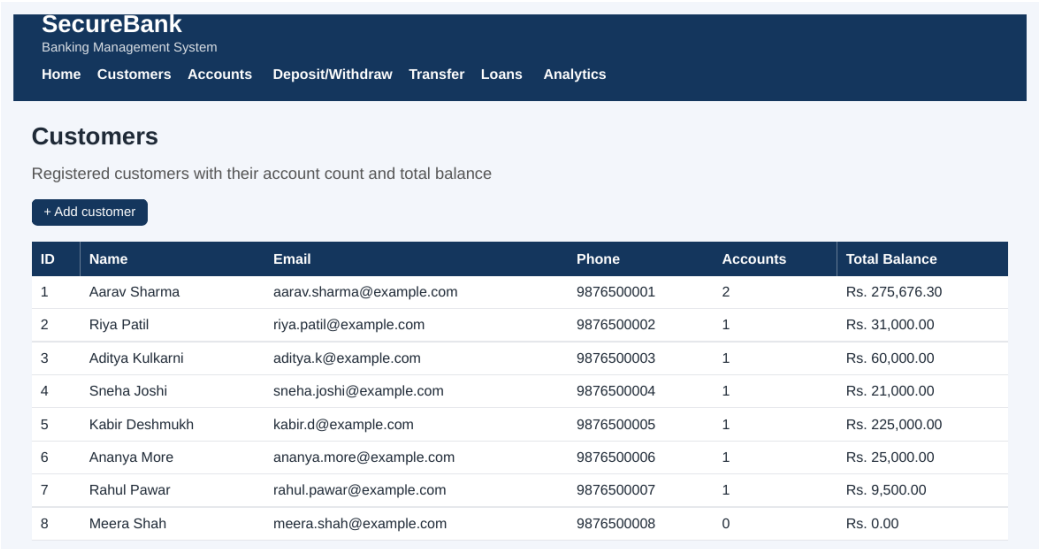


Figure 3: Customers page (LEFT JOIN with GROUP BY)

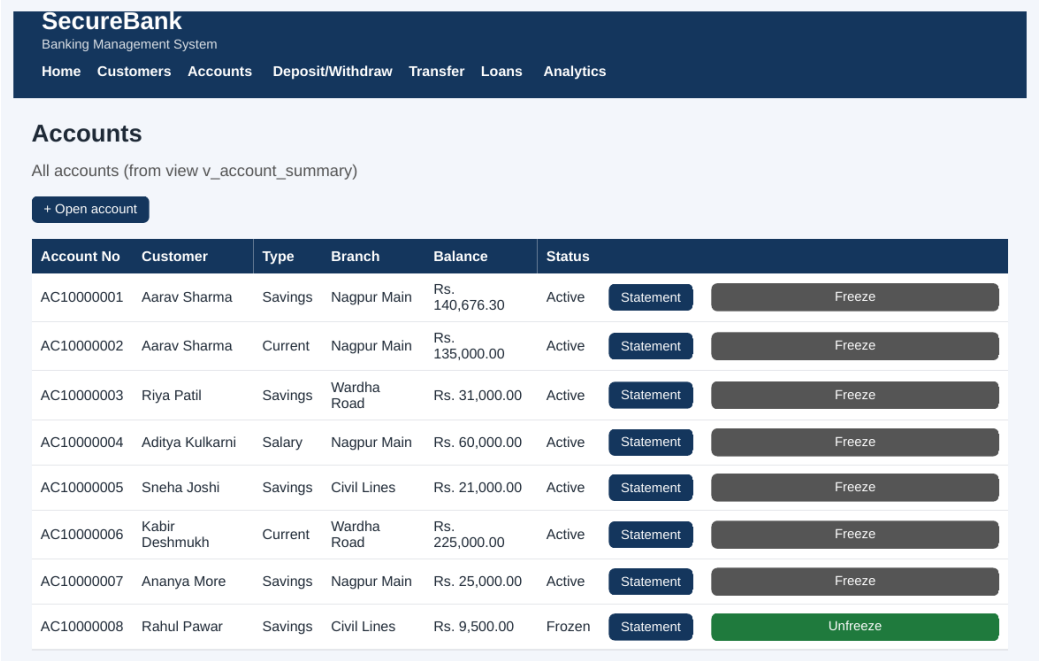


Figure 4: Accounts page (from view v_account_summary)

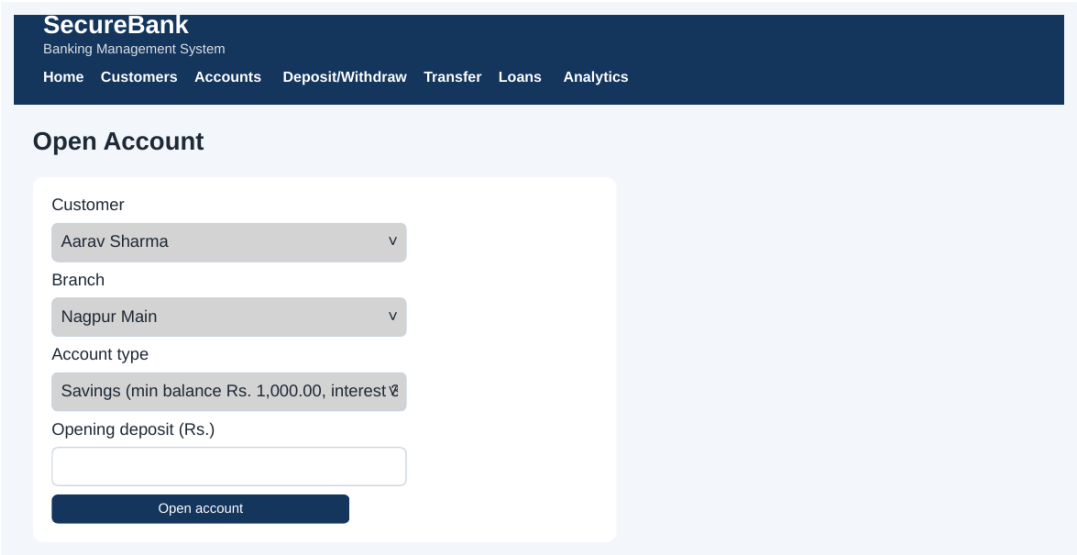


Figure 5: Open Account form

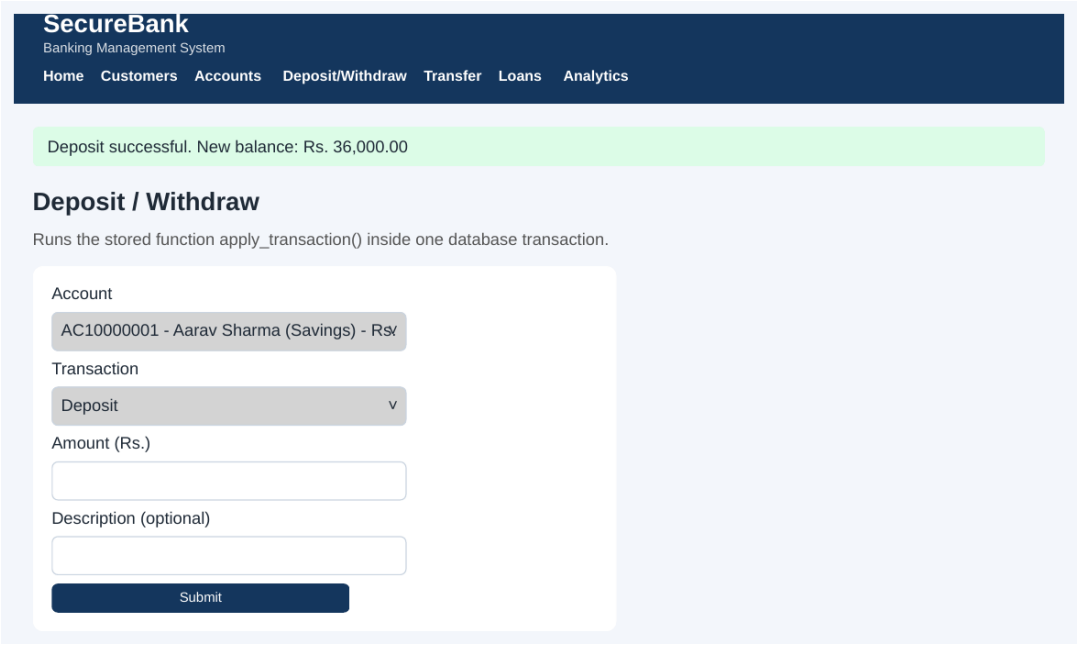


Figure 6: Deposit of Rs. 5,000 completed successfully

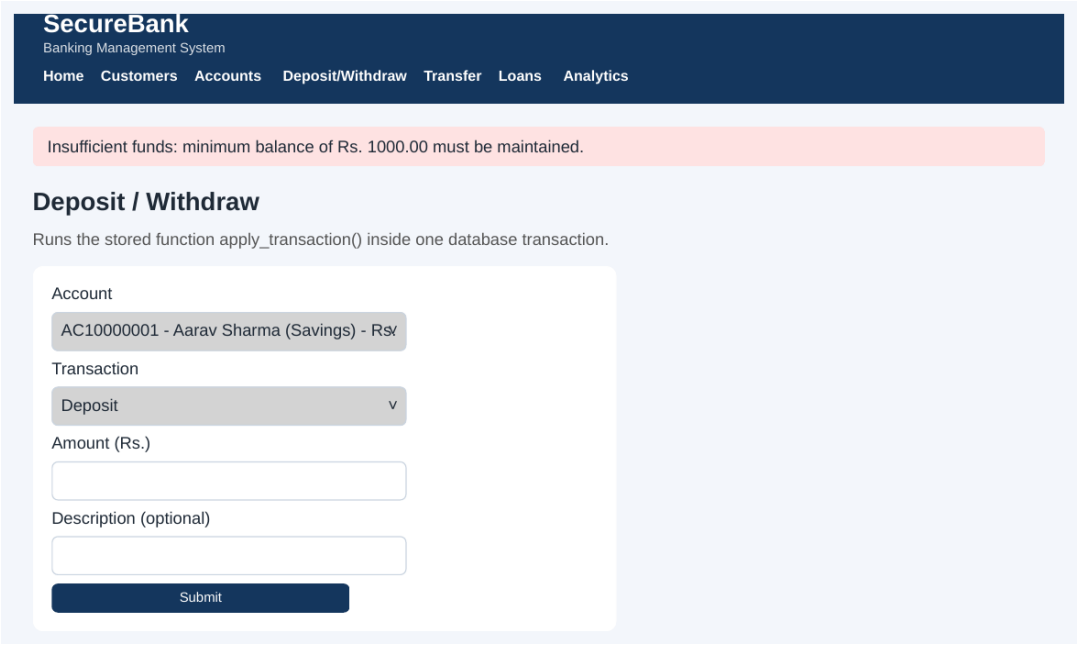


Figure 7: Withdrawal rejected: minimum balance rule enforced by the stored function

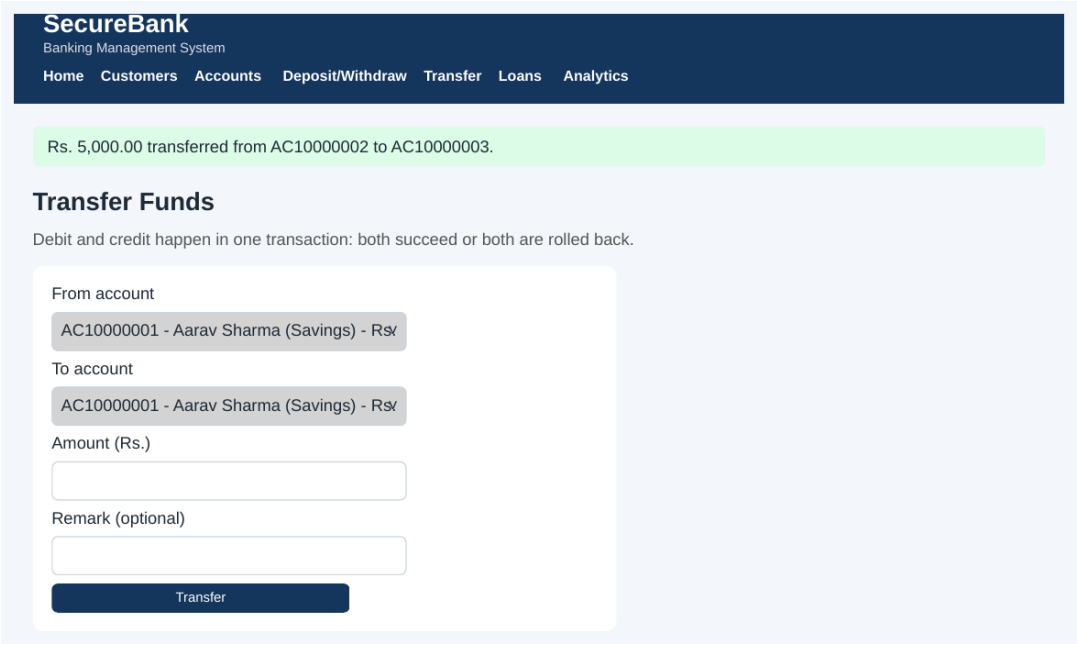


Figure 8: Funds transfer completed (debit and credit in one transaction)

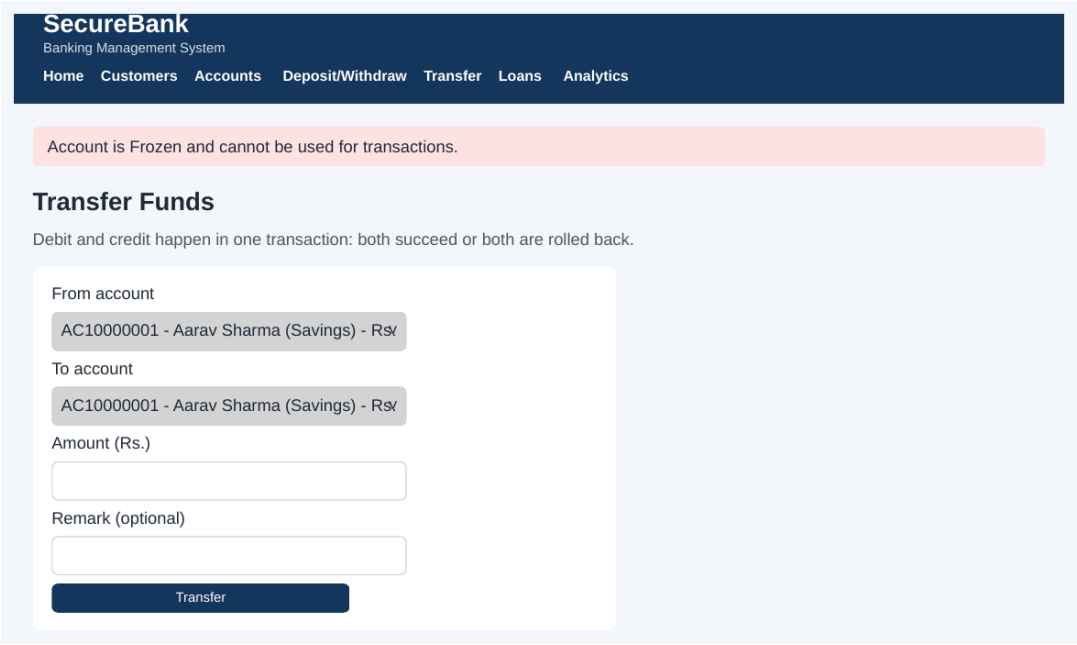


Figure 9: Transfer to a frozen account rejected and rolled back

SecureBank
Banking Management System

Home Customers Accounts Deposit/Withdraw Transfer Loans Analytics

Statement: AC10000003

Riya Patil | Savings account | Wardha Road branch | Balance Rs. 41,000.00 | Active

Txn ID	Time	Type	Amount	Balance After	Description
24	2026-10-07 03:12	Transfer-In	Rs. 5,000.00	Rs. 41,000.00	Transfer from AC10000002. Rent
22	2026-10-07 03:12	Deposit	Rs. 5,000.00	Rs. 36,000.00	Cash deposit
16	2026-10-07 03:12	Transfer-In	Rs. 3,000.00	Rs. 31,000.00	Transfer from AC10000001
10	2026-10-07 03:12	Withdrawal	Rs. 2,000.00	Rs. 28,000.00	ATM withdrawal
3	2026-10-07 03:12	Deposit	Rs. 30,000.00	Rs. 30,000.00	Opening deposit

Figure 10: Account statement of AC10000003

SecureBank
Banking Management System

Home Customers Accounts Deposit/Withdraw Transfer Loans Analytics

EMI of Rs. 6,227.51 paid for loan 2.

Loans

Loan applications, approval and EMI payments (EMI from stored function calc_emi)

+ Apply for loan

ID	Customer	Type	Principal	Rate	Months	EMI	Status	EMIs Paid	
3	Kabir Deshmukh	Vehicle	Rs. 500,000.00	9.50%	48	Rs. 12,561.57	Rejected	0/48	
2	Aditya Kulkarni	Education	Rs. 300,000.00	9.00%	60	Rs. 6,227.51	Approved	1/60	Pay EMI
1	Aarav Sharma	Personal	Rs. 120,000.00	12.00%	12	Rs. 10,661.85	Approved	2/12	Pay EMI

Figure 11: Loans page after approving loan 2 and paying one EMI

SecureBank
Banking Management System

Home Customers Accounts Deposit/Withdraw Transfer Loans Analytics

Monthly interest credited to 5 accounts.

8 Customers

7 Active accounts

Rs. 947,497.28 Total deposits

2 Approved loans

Recent transactions

ID	Account	Customer	Type	Amount	Balance After	Time
32	AC10000004	Aditya Kulkarni	Interest	Rs. 884.43	Rs. 354,656.92	2026-10-07 03:12
31	AC10000005	Sneha Joshi	Interest	Rs. 61.25	Rs. 21,061.25	2026-10-07 03:12
30	AC10000007	Ananya More	Interest	Rs. 72.92	Rs. 25,072.92	2026-10-07 03:12
29	AC10000001	Aarav Sharma	Interest	Rs. 410.31	Rs. 141,086.61	2026-10-07 03:12
28	AC10000003	Riya Patil	Interest	Rs. 119.58	Rs. 41,119.58	2026-10-07 03:12
27	AC10000004	Aditya Kulkarni	EMI	Rs. 6,227.51	Rs. 353,772.49	2026-10-07 03:12
26	AC10000004	Aditya Kulkarni	Loan Disbursal	Rs. 300,000.00	Rs. 360,000.00	2026-10-07 03:12
24	AC10000003	Riya Patil	Transfer-In	Rs. 5,000.00	Rs. 41,000.00	2026-10-07 03:12

Post monthly interest

Figure 12: Monthly interest credited to 5 accounts



Figure 13: Analytics page: views, aggregates, subquery, NOT EXISTS and trigger audit log

Database output (PostgreSQL / psql)

```
List of relations
 Schema |     Name      | Type  | Owner
--------+---------------+-------+-------
 public | account_types | table | bank
 public | accounts      | table | bank
 public | audit_log     | table | bank
 public | branches      | table | bank
 public | customers     | table | bank
 public | loan_payments | table | bank
 public | loans         | table | bank
 public | transactions  | table | bank
(8 rows)
```

Figure 14: Tables created in the securebank database (8 tables)

```
table_name   | record_count
--------------+--------------
 branches     |            3
 customers    |            8
 account_types |            3
 accounts     |            8
 transactions  |           21
 loans        |            3
 loan_payments |            2
 audit_log    |            9
(8 rows)
```

Figure 15: Database record count query output (sample data)

```
indexname       |   tablename
-----------------------+--------------
 idx_accounts_branch   | accounts
 idx_accounts_customer | accounts
 idx_accounts_type     | accounts
 idx_audit_account     | audit_log
 idx_loans_account     | loans
 idx_loans_customer    | loans
 idx_payments_loan     | loan_payments
 idx_txn_account       | transactions
 idx_txn_time          | transactions
(9 rows)
```

Figure 16: Indexes created on the database

```
function_name     | returns
----------------------+---------
 apply_transaction     | numeric
 calc_emi              | numeric
 log_account_status    | trigger
 post_monthly_interest | integer
(4 rows)
```

Figure 17: Stored functions in the database

```
emi_120000_12pct_12m
---------------------
            10661.85
(1 row)
```

Figure 18: calc_emi() output for Rs. 1,20,000 at 12% for 12 months

```
account_number | old_status | new_status |    changed_at
---------------+------------+------------+-----------------
 AC10000001    |            | Active     | 2026-10-07 03:12
 AC10000002    |            | Active     | 2026-10-07 03:12
 AC10000003    |            | Active     | 2026-10-07 03:12
 AC10000004    |            | Active     | 2026-10-07 03:12
 AC10000005    |            | Active     | 2026-10-07 03:12
 AC10000006    |            | Active     | 2026-10-07 03:12
 AC10000007    |            | Active     | 2026-10-07 03:12
 AC10000008    |            | Active     | 2026-10-07 03:12
 AC10000008    | Active     | Frozen     | 2026-10-07 03:12
(9 rows)
```

Figure 19: Audit log filled by the trigger (account opened and AC10000008 frozen)

```
txn_id |    txn_type    |  amount   | balance_after |   description
-------+----------------+-----------+---------------+-----------------
      4 | Deposit        |  45000.00 |      45000.00 | Salary credit
     11 | Deposit        |  15000.00 |      60000.00 | Bonus
     26 | Loan Disbursal | 300000.00 |     360000.00 | Loan 2 disbursed
     27 | EMI            |   6227.51 |     353772.49 | EMI loan 2
     32 | Interest       |    884.43 |     354656.92 | Monthly interest
(5 rows)
```

Figure 20: Statement rows of account 4 (salary, bonus, loan disbursal, EMI and interest)

```
account_id | account_number | balance  | status
-----------+----------------+----------+--------
         3 | AC10000003     | 41119.58 | Active
         8 | AC10000008     |  9500.00 | Frozen
(2 rows)
```

Figure 21: Balances after the failed transfer: AC10000003 changed only by earlier successful operations; frozen AC10000008 unchanged

```
ERROR:  Insufficient funds: minimum balance of Rs. 1000.00 must be maintained.
CONTEXT:  PL/pgSQL function apply_transaction(integer,character varying,numeric,text,integer) line 17 at RAISE
```

Figure 22: Stored function raising an error for an invalid withdrawal

```
ERROR:  new row for relation "accounts" violates check constraint "accounts_balance_check"
DETAIL:  Failing row contains (1, AC10000001, 1, 1, 1, -500.00, Active, 2026-10-07 03:12:34.508297).
```

Figure 23: CHECK constraint rejecting a negative balance

10. Testing

The project includes eleven automated tests (Python unittest). They reload the schema and sample data and test the application against the real PostgreSQL database. All tests passed.

Test	What it verifies	Result
test_pages_load	All 11 pages (home, customers, accounts, statement, forms, loans, analytics) load successfully.	Pass
test_sample_data_balances	Loaded balances match hand calculation (for example 50000 - 5000 - 3000 + 120000 - 2 EMIs = 140676.30).	Pass
test_deposit_and_withdraw	Deposit and withdrawal update the balance correctly.	Pass
test_insufficient_funds_rolls_back	A too-large withdrawal is rejected; balance and transaction count are unchanged.	Pass
test_minimum_balance_enforced	Withdrawal leaving less than the minimum balance fails; leaving exactly the minimum succeeds.	Pass
test_transfer_success_and_atomic_rollback	Transfer updates both balances; a transfer to a frozen account leaves all balances unchanged (rollback).	Pass
test_frozen_account_and_audit_trigger	Freezing writes an audit row via the trigger and blocks transactions on that account.	Pass
test_loan_approve_and_emi	Approval credits the principal; paying an EMI debits the account and records the payment.	Pass
test_emi_function	calc_emi(120000, 12, 12) = 10661.85 and the zero-interest case works.	Pass
test_monthly_interest	Interest is posted to exactly the 5 active interest-bearing accounts.	Pass
test_constraints	Under-18 customer and negative balance are rejected by CHECK constraints.	Pass

Command used: `python -m unittest discover tests`

11. Conclusion

SecureBank successfully demonstrates the practical use of a relational database for banking. The project integrates PostgreSQL with a Flask web application and provides customer and account management, deposits, withdrawals, transfers, loans with EMI payments, interest posting and analytics.

The design demonstrates important DBMS concepts including keys, constraints, normalization, a sequence, joins, aggregate queries, subqueries, indexing, views, stored functions, a trigger and atomic transactions with row locking. Keeping the banking rules inside the database makes the data safe even if the application code changes. The project provides a foundation that can be extended with security and reporting features.

12. Future Scope

- Add secure login with roles (customer, teller, manager) and CSRF protection.
- Add a customer self-service portal and PDF account statements.
- Schedule monthly interest posting automatically.
- Add debit cards, UPI-style payments and standing instructions.
- Add EMI schedules and loan prepayment.
- Add fraud-detection rules for unusual transactions and dashboards for managers.

Appendix A: Current Database Record Summary

Table	Current Records
branches	3
customers	8
account_types	3
accounts	8
transactions	21
loans	3
loan_payments	2
audit_log	9

(Record counts of the sample data supplied with the project.)

Appendix B: Complete Database Schema (SQL)

```
-- SecureBank: Banking Management System (PostgreSQL)
DROP VIEW IF EXISTS v_account_summary, v_branch_summary, v_loan_summary CASCADE;
DROP TABLE IF EXISTS audit_log, loan_payments, loans, transactions, accounts, account_types, customers,
    branches CASCADE;
DROP FUNCTION IF EXISTS apply_transaction(INT, VARCHAR, NUMERIC, TEXT, INT), post_monthly_interest(),
    calc_emi(NUMERIC, NUMERIC, INT), log_account_status() CASCADE;
DROP SEQUENCE IF EXISTS account_no_seq;

CREATE SEQUENCE account_no_seq START 10000001;

CREATE TABLE branches (
    branch_id    SERIAL PRIMARY KEY,
    branch_name  VARCHAR(60) NOT NULL UNIQUE,
    ifsc_code    VARCHAR(11) NOT NULL UNIQUE CHECK (ifsc_code ~ '^[A-Z]{4}0[A-Z0-9]{6}$'),
    city         VARCHAR(40) NOT NULL
);

CREATE TABLE customers (
    customer_id  SERIAL PRIMARY KEY,
    full_name    VARCHAR(80) NOT NULL,
    email        VARCHAR(80) NOT NULL UNIQUE CHECK (email LIKE '%_@_%.__%'),
    phone        VARCHAR(10) NOT NULL UNIQUE CHECK (phone ~ '^[6-9][0-9]{9}$'),
    dob          DATE NOT NULL CHECK (dob <= DATE '2100-01-01'),
    address      TEXT,
    created_at   TIMESTAMP NOT NULL DEFAULT NOW(),
    CHECK (dob <= CURRENT_DATE - INTERVAL '18 years')          -- customers must be adults
);

CREATE TABLE account_types (
    type_id        SERIAL PRIMARY KEY,
    type_name      VARCHAR(20) NOT NULL UNIQUE,
    min_balance    NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (min_balance >= 0),
    interest_rate  NUMERIC(4,2) NOT NULL DEFAULT 0 CHECK (interest_rate BETWEEN 0 AND 15)
);

CREATE TABLE accounts (
    account_id      SERIAL PRIMARY KEY,
    account_number  VARCHAR(12) NOT NULL UNIQUE DEFAULT ('AC' || LPAD(nextval('account_no_seq')::TEXT, 8,
        '0')),
    customer_id     INT NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT,
    branch_id       INT NOT NULL REFERENCES branches(branch_id),
    type_id         INT NOT NULL REFERENCES account_types(type_id),
    balance         NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (balance >= 0),
    status          VARCHAR(10) NOT NULL DEFAULT 'Active' CHECK (status IN ('Active','Frozen','Closed')),
    opened_at       TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE transactions (
    txn_id             SERIAL PRIMARY KEY,
    account_id         INT NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
    txn_type           VARCHAR(15) NOT NULL
        CHECK (txn_type IN ('Deposit','Withdrawal','Transfer-In','Transfer-Out','Interest','Loan
            Disbursal','EMI')),
    amount             NUMERIC(14,2) NOT NULL CHECK (amount > 0),
    balance_after      NUMERIC(14,2) NOT NULL CHECK (balance_after >= 0),
    related_account_id INT REFERENCES accounts(account_id),
    description        TEXT,
    txn_time           TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE loans (
    loan_id        SERIAL PRIMARY KEY,
    customer_id    INT NOT NULL REFERENCES customers(customer_id),
    account_id     INT NOT NULL REFERENCES accounts(account_id),
    loan_type      VARCHAR(15) NOT NULL CHECK (loan_type IN ('Home','Education','Personal','Vehicle')),
    principal      NUMERIC(14,2) NOT NULL CHECK (principal > 0),
    annual_rate    NUMERIC(4,2) NOT NULL CHECK (annual_rate >= 0),
    tenure_months  INT NOT NULL CHECK (tenure_months BETWEEN 1 AND 360),
    status         VARCHAR(10) NOT NULL DEFAULT 'Pending' CHECK (status IN
        ('Pending','Approved','Rejected','Closed')),
    applied_at     TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE loan_payments (
    payment_id  SERIAL PRIMARY KEY,
    loan_id     INT NOT NULL REFERENCES loans(loan_id) ON DELETE CASCADE,
    amount      NUMERIC(12,2) NOT NULL CHECK (amount > 0),
    paid_at     TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE audit_log (
    log_id      SERIAL PRIMARY KEY,
    account_id  INT NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
    old_status  VARCHAR(10),
    new_status  VARCHAR(10) NOT NULL,
    changed_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Indexes on foreign keys and frequent filters
CREATE INDEX idx_accounts_customer ON accounts(customer_id);
CREATE INDEX idx_accounts_branch   ON accounts(branch_id);
CREATE INDEX idx_accounts_type     ON accounts(type_id);
CREATE INDEX idx_txn_account       ON transactions(account_id);
CREATE INDEX idx_txn_time          ON transactions(txn_time);
CREATE INDEX idx_loans_customer    ON loans(customer_id);
CREATE INDEX idx_loans_account     ON loans(account_id);
CREATE INDEX idx_payments_loan     ON loan_payments(loan_id);
CREATE INDEX idx_audit_account     ON audit_log(account_id);

-- Stored function: the ONLY way balances change (validates, updates balance, writes the statement row)
CREATE FUNCTION apply_transaction(p_account INT, p_type VARCHAR, p_amount NUMERIC,
                                  p_desc TEXT DEFAULT NULL, p_related INT DEFAULT NULL)
RETURNS NUMERIC AS $$
DECLARE v_bal NUMERIC; v_status VARCHAR; v_min NUMERIC; v_new NUMERIC;
BEGIN
    IF p_amount IS NULL OR p_amount <= 0 THEN
        RAISE EXCEPTION 'Amount must be greater than zero.';
    END IF;
    SELECT a.balance, a.status, t.min_balance INTO v_bal, v_status, v_min
    FROM accounts a JOIN account_types t ON t.type_id = a.type_id
    WHERE a.account_id = p_account FOR UPDATE OF a;                  -- row lock
    IF NOT FOUND THEN RAISE EXCEPTION 'Account not found.'; END IF;
    IF v_status <> 'Active' THEN
        RAISE EXCEPTION 'Account is % and cannot be used for transactions.', v_status;
    END IF;
    IF p_type IN ('Withdrawal','Transfer-Out','EMI') THEN
        v_new := v_bal - p_amount;
        IF v_new < v_min THEN
            RAISE EXCEPTION 'Insufficient funds: minimum balance of Rs. % must be maintained.', v_min;
        END IF;
    ELSE
        v_new := v_bal + p_amount;
    END IF;
    UPDATE accounts SET balance = v_new WHERE account_id = p_account;
    INSERT INTO transactions (account_id, txn_type, amount, balance_after, related_account_id,
        description)
    VALUES (p_account, p_type, p_amount, v_new, p_related, p_desc);
    RETURN v_new;
END;
$$ LANGUAGE plpgsql;

-- EMI = P * r * (1+r)^n / ((1+r)^n - 1), r = monthly rate
CREATE FUNCTION calc_emi(p NUMERIC, annual_rate NUMERIC, months INT) RETURNS NUMERIC AS $$
    SELECT CASE WHEN annual_rate = 0 THEN ROUND(p / months, 2)
           ELSE ROUND(p * (annual_rate/1200) * POWER(1 + annual_rate/1200, months)
                      / (POWER(1 + annual_rate/1200, months) - 1), 2) END;
$$ LANGUAGE sql IMMUTABLE;

-- Credits monthly interest to every active account whose type earns interest
CREATE FUNCTION post_monthly_interest() RETURNS INT AS $$
DECLARE r RECORD; n INT := 0; v_int NUMERIC;
BEGIN
    FOR r IN SELECT a.account_id, a.balance, t.interest_rate
             FROM accounts a JOIN account_types t ON t.type_id = a.type_id
             WHERE a.status = 'Active' AND t.interest_rate > 0 LOOP
        v_int := ROUND(r.balance * r.interest_rate / 1200, 2);
        IF v_int > 0 THEN
            PERFORM apply_transaction(r.account_id, 'Interest', v_int, 'Monthly interest');
            n := n + 1;
        END IF;
    END LOOP;
    RETURN n;
END;
$$ LANGUAGE plpgsql;

-- Trigger: audit trail for account status (opened / frozen / unfrozen / closed)
CREATE FUNCTION log_account_status() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log (account_id, old_status, new_status) VALUES (NEW.account_id, NULL,
            NEW.status);
    ELSIF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO audit_log (account_id, old_status, new_status) VALUES (NEW.account_id, OLD.status,
```

```
            NEW.status);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_account_status AFTER INSERT OR UPDATE OF status ON accounts
FOR EACH ROW EXECUTE FUNCTION log_account_status();

-- Views
CREATE VIEW v_account_summary AS
SELECT a.account_id, a.account_number, c.full_name, t.type_name, b.branch_name, a.balance, a.status,
    a.opened_at
FROM accounts a
JOIN customers c ON c.customer_id = a.customer_id
JOIN account_types t ON t.type_id = a.type_id
JOIN branches b ON b.branch_id = a.branch_id;

CREATE VIEW v_branch_summary AS
SELECT b.branch_name, b.city, COUNT(a.account_id) AS accounts, COALESCE(SUM(a.balance), 0) AS
    total_balance
FROM branches b LEFT JOIN accounts a ON a.branch_id = b.branch_id
GROUP BY b.branch_id;

CREATE VIEW v_loan_summary AS
SELECT l.loan_id, c.full_name, a.account_number, l.loan_type, l.principal, l.annual_rate, l.tenure_months,
       calc_emi(l.principal, l.annual_rate, l.tenure_months) AS emi, l.status,
       COUNT(p.payment_id) AS emis_paid, l.applied_at
FROM loans l
JOIN customers c ON c.customer_id = l.customer_id
JOIN accounts a ON a.account_id = l.account_id
LEFT JOIN loan_payments p ON p.loan_id = l.loan_id
GROUP BY l.loan_id, c.full_name, a.account_number;
```